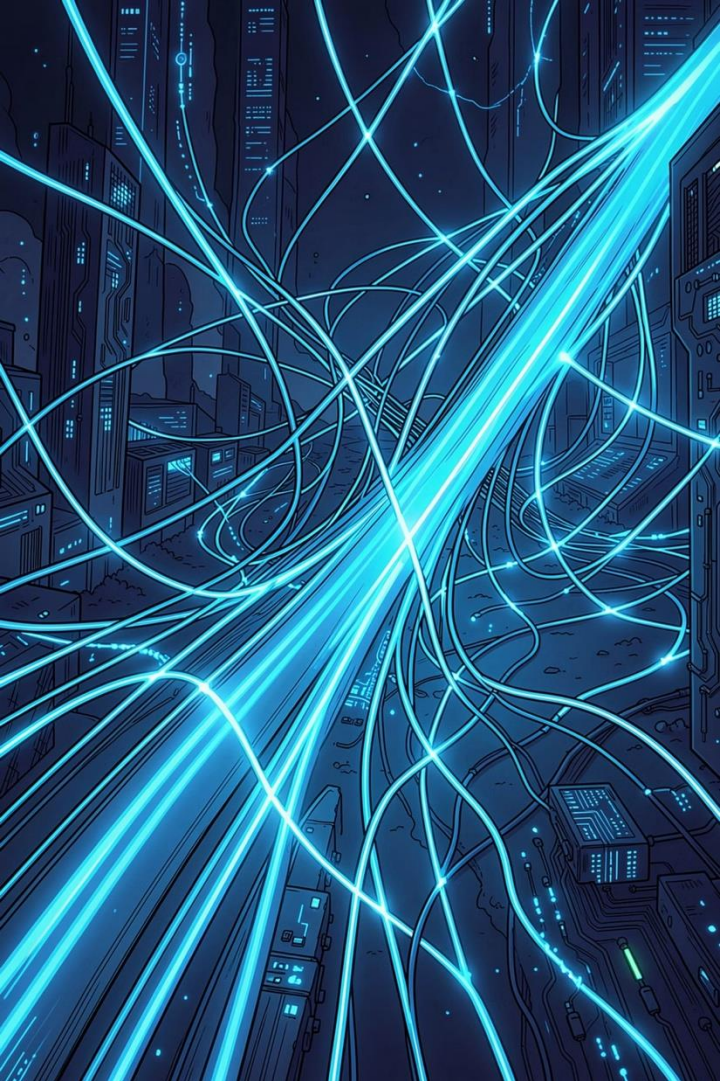


Mastering Workflow Orchestration

A Data Engineer's Guide to RAG on Airflow

SOMYOST PHETSAIKAEW

About the Instructor

Somyost Phetsaikaew

Deputy Head of Data Management at Auto X

The Raw Data Deluge

175ZB

Global Data by 2025

Zettabytes generated across every industry and device

80%

Unstructured Data

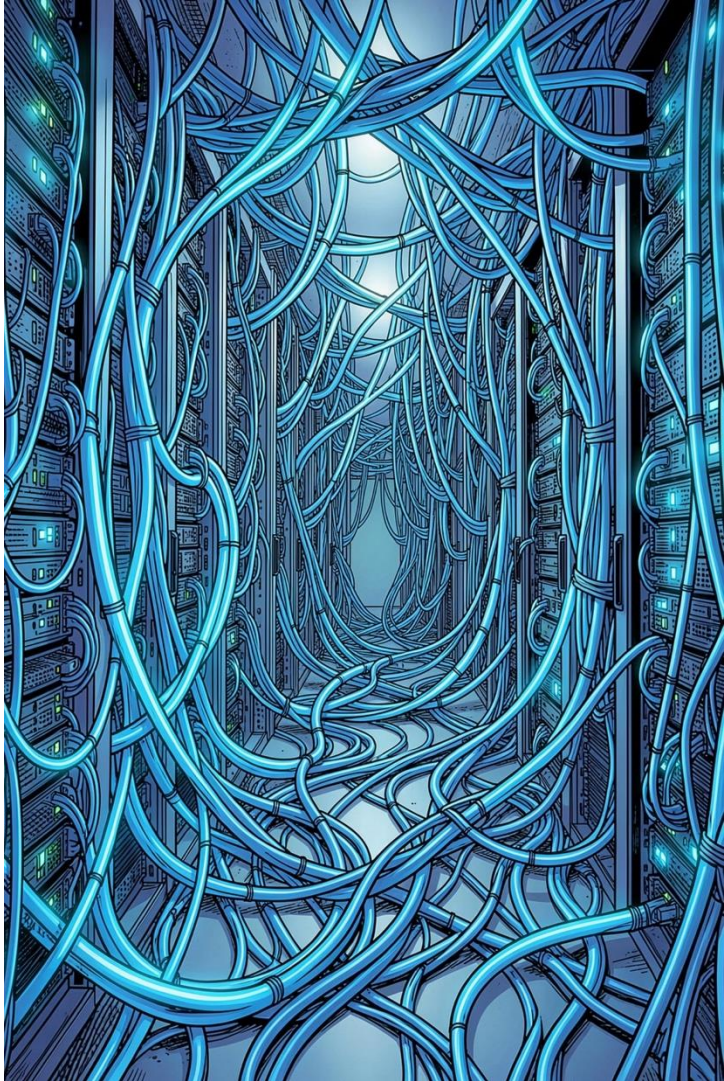
Logs, IoT sensors, and API streams with no defined schema

3

Core Challenges

Data exists — but remains fragmented, noisy, and unusable

Businesses are drowning in raw data. The gap between **having data** and **using data** is where Data Engineers operate.



ROLE DEFINITION

Defining the Data Engineer

A Data Engineer is not just a coder — they are the **architect of reliable data ecosystems**. Their mission: transform raw, chaotic inputs into trusted, scalable assets that power decisions at scale.

Designer

Builds pipelines, not just scripts

Bridge

Connects infrastructure to intelligence

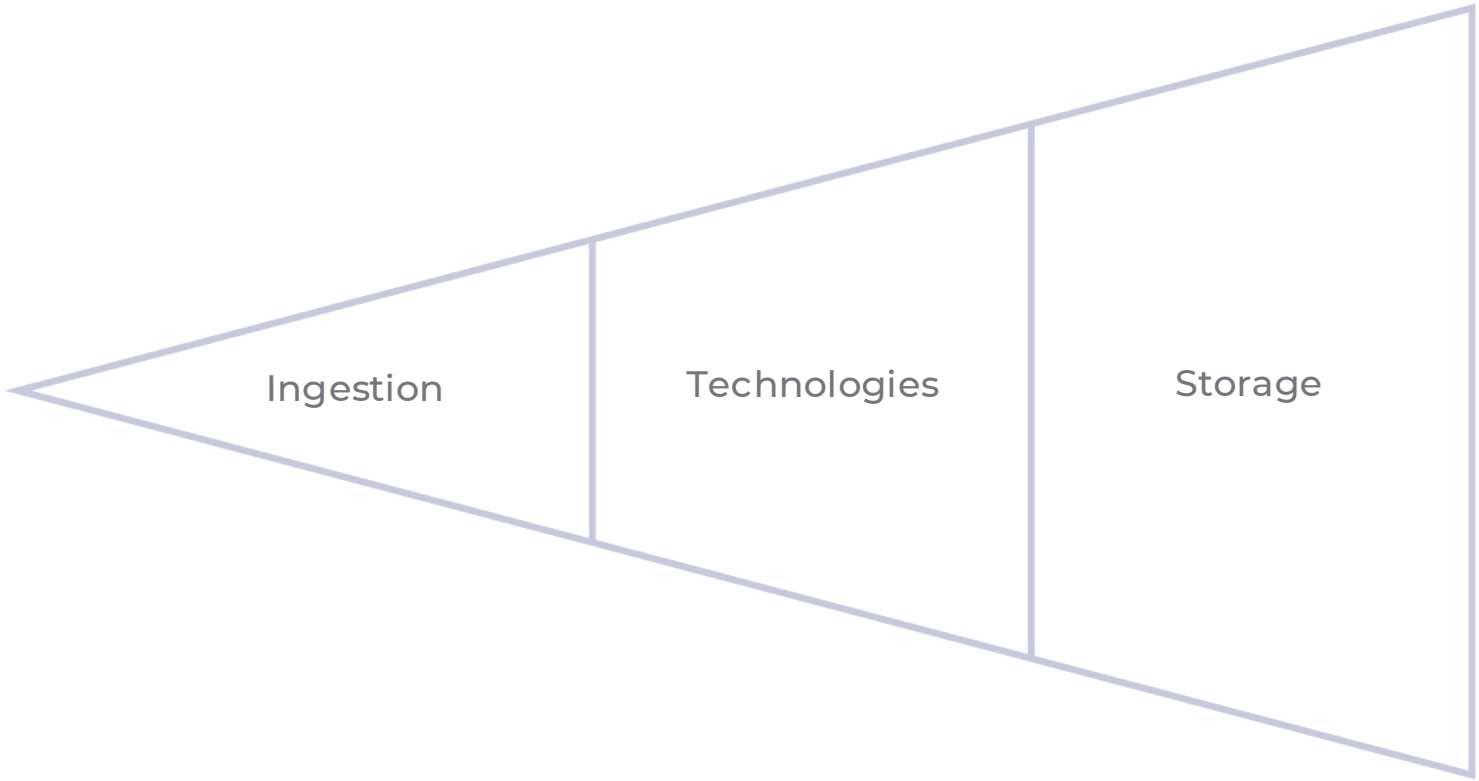
Guardian

Ensures data quality and trust

LIFECYCLE · PHASE I

Ingestion & Storage

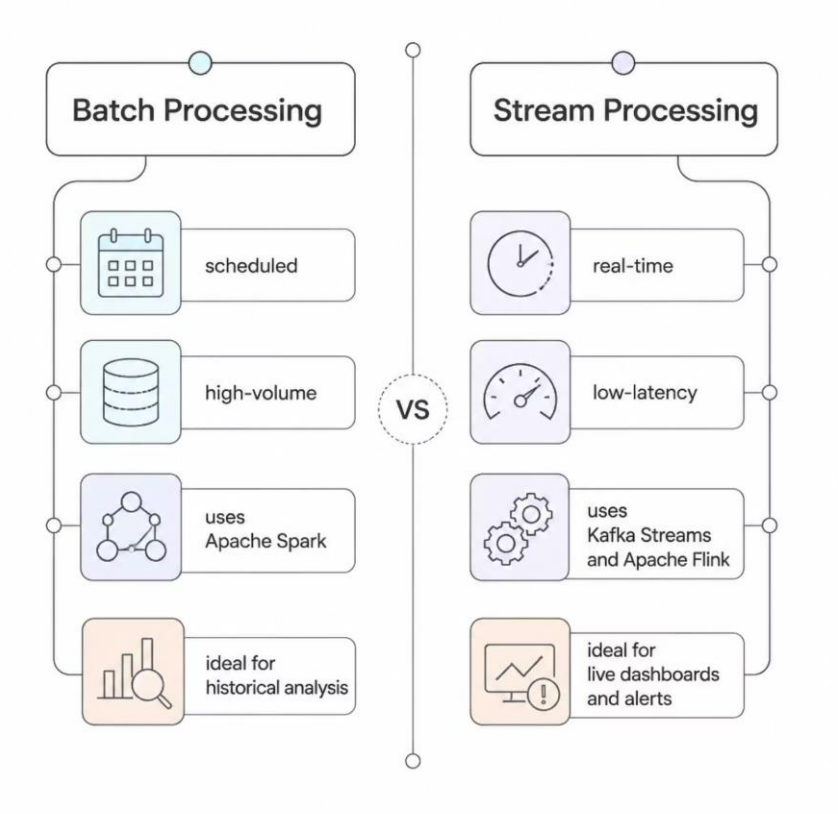
The first phase of any data pipeline: collecting data from diverse sources and deciding where it lives.



Data Engineers choose the right storage strategy — **Data Lakes** for raw volume, **Warehouses** for structured analytics, or **Lakehouses** for the best of both worlds.

LIFECYCLE · PHASE II

Processing Power



Choosing the Right Engine

Not all data needs to move at the speed of light. Data Engineers design systems that match processing strategy to business need.

Apache Spark
Massive distributed batch workloads

Kafka Streams
Real-time event streaming at scale

Apache Flink
Stateful stream processing with low latency

The Transformation Engine

ETL & ELT Pipelines

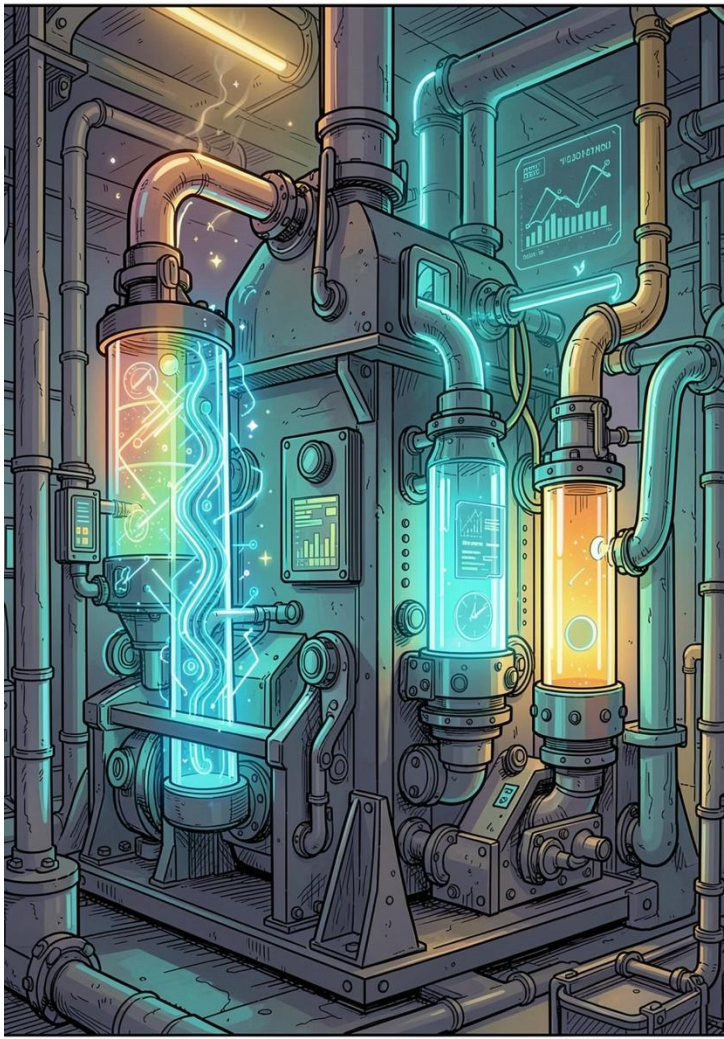
Cleaning, enriching, and structuring raw data into analysis-ready formats. Choose **ETL** for pre-load transformation or **ELT** for cloud-native flexibility.

Orchestration

Tools like **dbt** and **Apache Airflow** automate the flow of intelligence — scheduling, monitoring, and recovering from failures automatically.

Quality Assurance

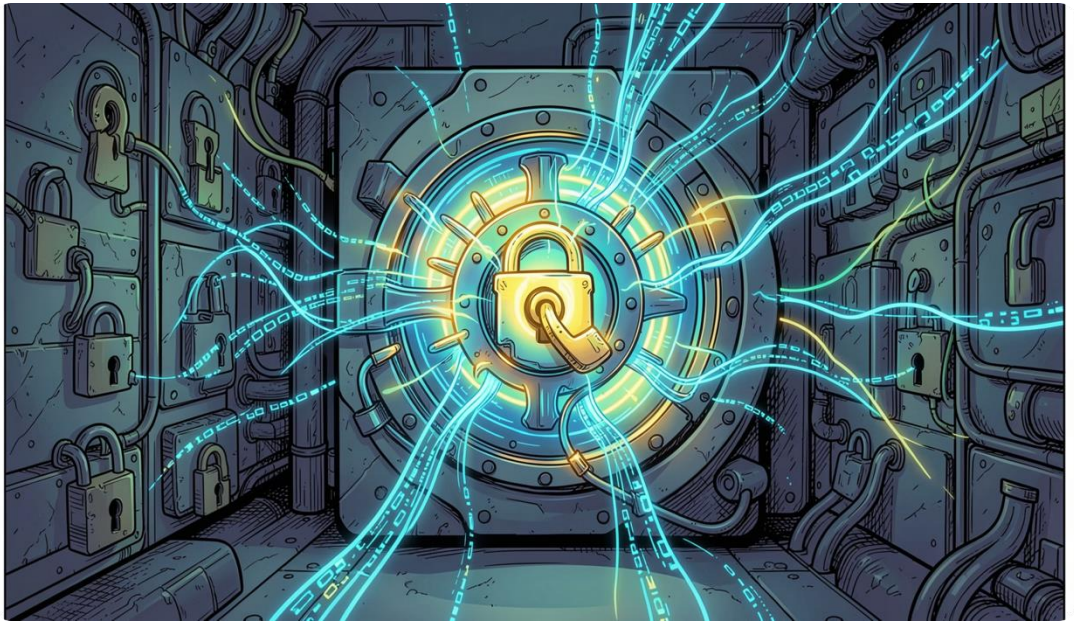
Automated tests and validation rules ensure data is accurate, complete, and consistent *before* it ever reaches a dashboard.



TRUST & COMPLIANCE

Governance & Trust

Data without governance is a liability. Data Engineers enforce the rules that make data safe, auditable, and compliant.



Schema Enforcement

Consistent structure, deduplication, and type validation across all pipelines

Security & Compliance

Navigating GDPR, HIPAA, and enterprise security protocols by design

Data Cataloging

Tools like **Alation** and **Collibra** track lineage, ownership, and metadata

The Evolving Role

Data Engineer vs. Data Scientist

The Data Engineer **builds the highway**; the Data Scientist **drives the car**. One enables, the other explores — both are essential.

The Modern Toolkit

The role has shifted from manual maintenance to **automated, serverless architectures** like AWS Glue.

Data Engineer

Infrastructure, pipelines, reliability

Data Scientist

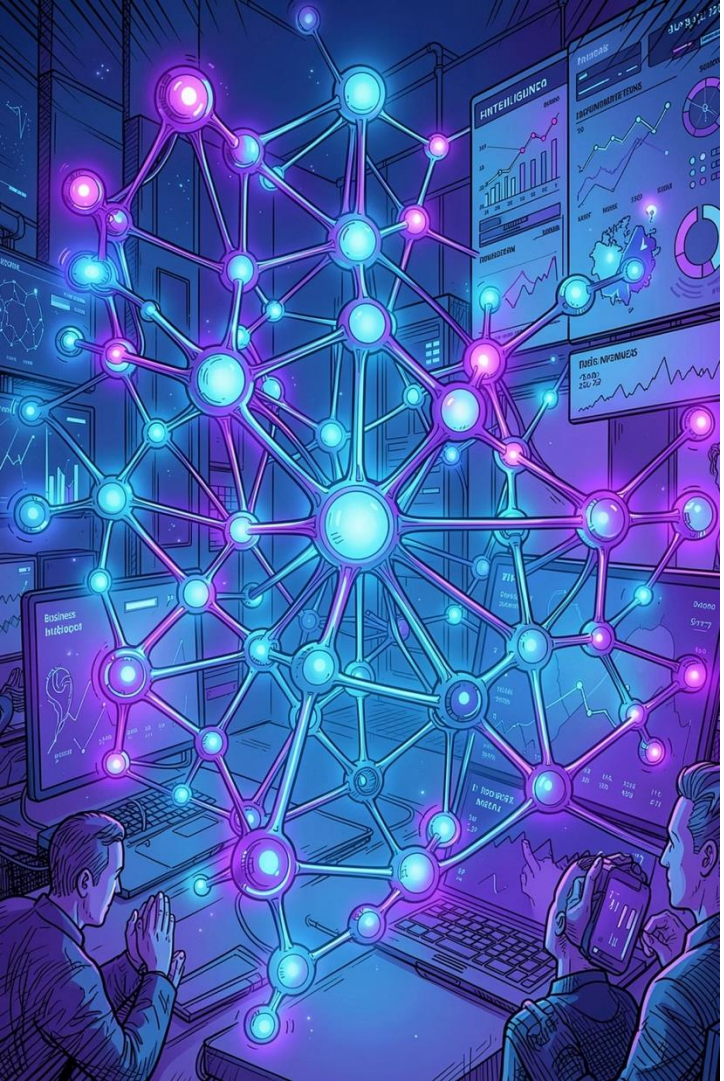
Models, analysis, insights

SQL

Python

Cloud Platforms

Distributed Computing



Modern Impact: From Insights to AI

Fueling ML

High-quality, feature-ready data is the foundation of every successful machine learning model

Business Intelligence

Enabling self-service analytics with **Tableau** and **Power BI** for non-technical stakeholders

Real-World Scale

E-commerce and finance rely on Data Engineers to process billions of transactions at light speed

What is Apache Airflow?

Open-Source Orchestration

Programmatically author, schedule, and monitor data pipelines at any scale.

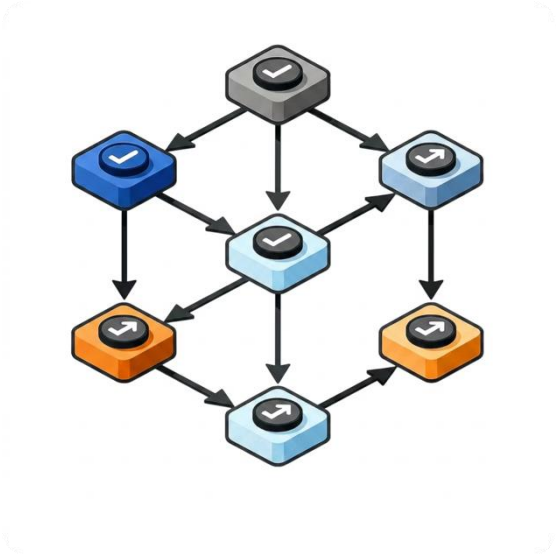
Born at Airbnb, 2014

Originally built to manage Airbnb's complex pipelines, donated to the Apache Software Foundation in 2016.

Pipelines as Code

Pipelines are defined in Python — version-controlled, testable, and fully collaborative across engineering teams.

The DAG: Heart of Airflow



Directed Acyclic Graph

A **DAG** is a collection of tasks with explicitly defined dependencies and execution order — the fundamental building block of every Airflow pipeline.

- **Directed** — tasks always flow in one direction, from upstream to downstream
- **Acyclic** — no circular loops allowed; each task runs to completion before triggering dependents
- **Python file** — each DAG is a plain Python script defining what runs, when, and in what order

Tasks, Operators & Sensors

Task

A single unit of work within a DAG — run a SQL query, call an API, or transform a dataset.

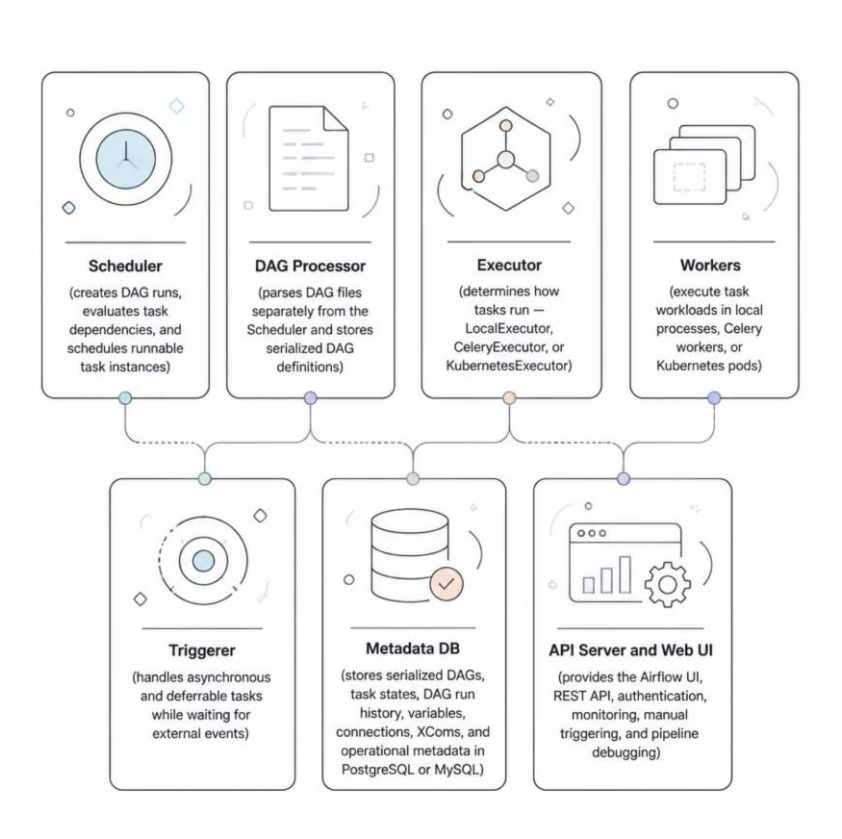
Operator

A reusable template defining *what* a task does. Choose from **BashOperator**, **PythonOperator**, **SparkSubmitOperator**, and 1,000+ community operators.

Sensor

A special operator that *waits* for a condition — a file arriving in S3, an HTTP endpoint returning 200, or a database record appearing.

Core Architecture



How the Components Work Together

Airflow's architecture separates concerns cleanly — the **DAG Processor** parses and serializes DAGs, the **Scheduler** orchestrates timing and dependencies, the **Executor** handles compute, the **Triggerer** manages deferrable and asynchronous tasks, the **Metadata DB** tracks state, and the **API Server** exposes everything through a unified UI and API. This modular design means you can swap executors from a single-node **LocalExecutor** up to a distributed **KubernetesExecutor** as your workloads scale — without rewriting your DAGs.

Key Features

Dynamic Pipelines

Generate DAGs programmatically using Python loops, config files, or templates — no static YAML required.

Rich UI

Graph view, Gantt chart, task logs, and full run history in one dashboard — debug and monitor at a glance.

Connections & Variables

Centralized secrets management for DB credentials, API keys, and environment configs — no hardcoded secrets.

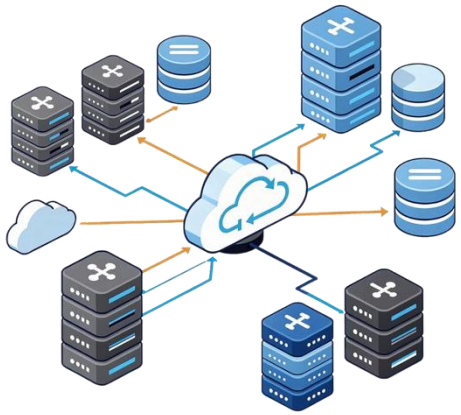
XComs

A lightweight mechanism that lets tasks share small pieces of data with each other across a DAG run.

Integrations & Ecosystem

80+ Official Provider Packages

Airflow's provider ecosystem covers virtually every tool in the modern data stack:



Managed Options

Astronomer and **Amazon MWAA** offer fully managed Airflow for teams that prefer not to self-host. Native integrations with **dbt**, **Great Expectations**, and **MLflow** enable end-to-end data pipelines out of the box.

1	2	3	4
Cloud	**Warehouses**	**Transforms**	**Infra**
AWS, GCP, Azure	Snowflake, BigQuery	dbt, Spark	Kubernetes, Slack

Real-World Use Cases

ETL / ELT Pipelines

Extract from APIs, transform with Spark or dbt, and load into Snowflake or BigQuery — all orchestrated end-to-end.

ML Pipelines

Automate model training, evaluation, and deployment on a recurring schedule with full lineage tracking.

Data Quality Checks

Run validation jobs after ingestion, blocking downstream consumers until data passes quality thresholds.

Report Generation

Trigger dashboards, send Slack alerts, or email reports automatically on pipeline completion.

What is an LLM — and What Can It Do?

What It Is

A **Large Language Model** takes Input → processes → returns Output in a single pass. It is fundamentally **stateless** — no memory persists between sessions.

Core Limitations

No memory between sessions. Cannot call external tools or APIs. Cannot make multi-step decisions autonomously.

Best Use Cases

Summarization, translation, and Q&A from a provided context window — tasks that fit within a single inference call.

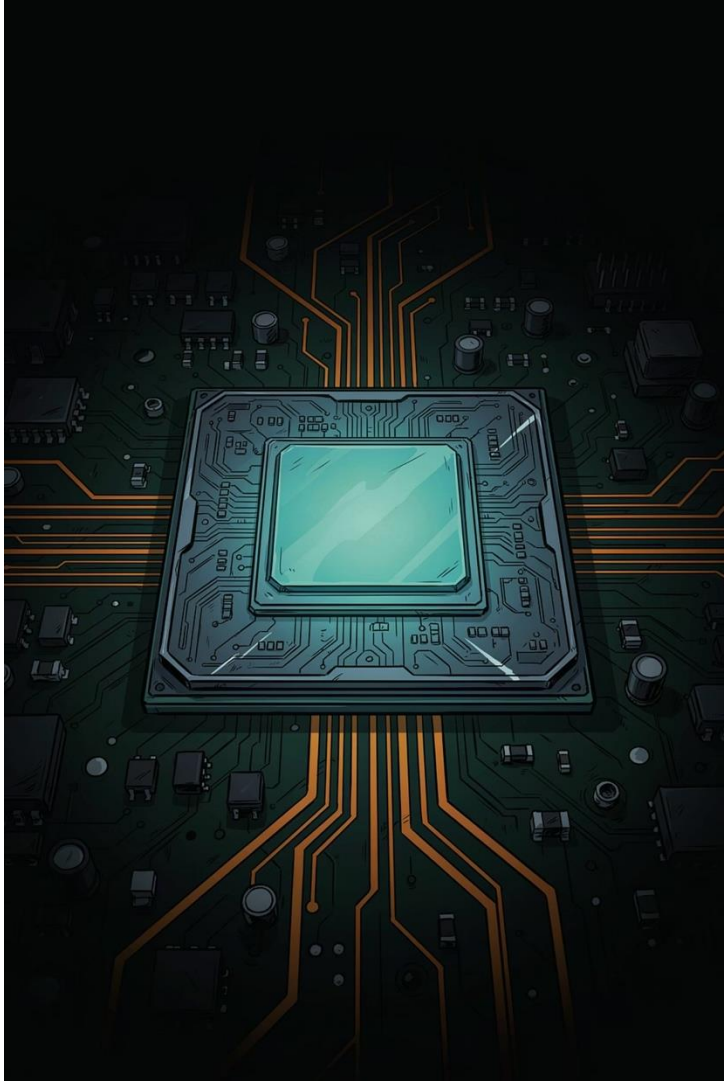
The One-Shot Mental Model

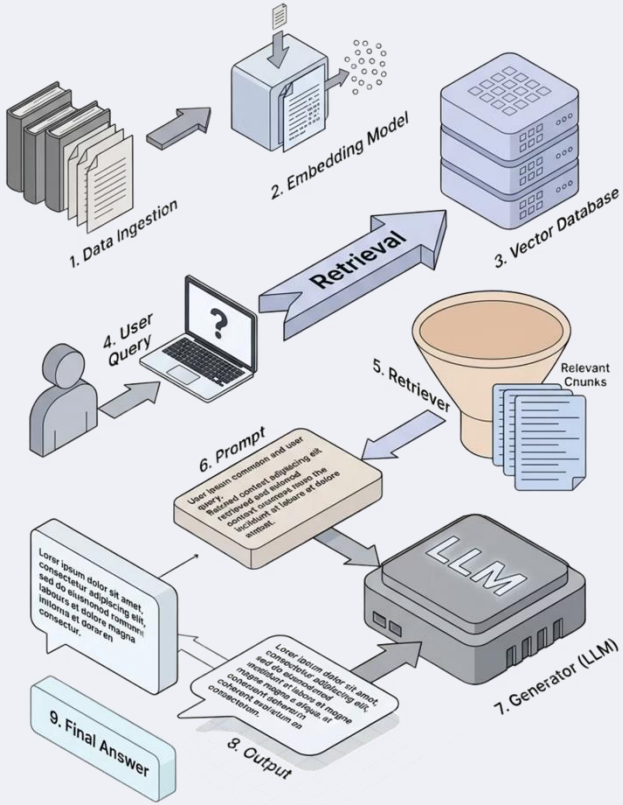
Think of an LLM as a very powerful **function call**:

- Input: prompt + context
- Process: transformer inference
- Output: generated text

Every call is independent. No state. No loops. No tools.

LLMs are the engine — not the vehicle.





What is RAG and Why Use It?

Retrieval-Augmented Generation — a technique that transforms the way LLMs answer questions, for developers who want to build AI systems that are truly accurate and up-to-date

Problems with LLMs Without RAG

Outdated Information

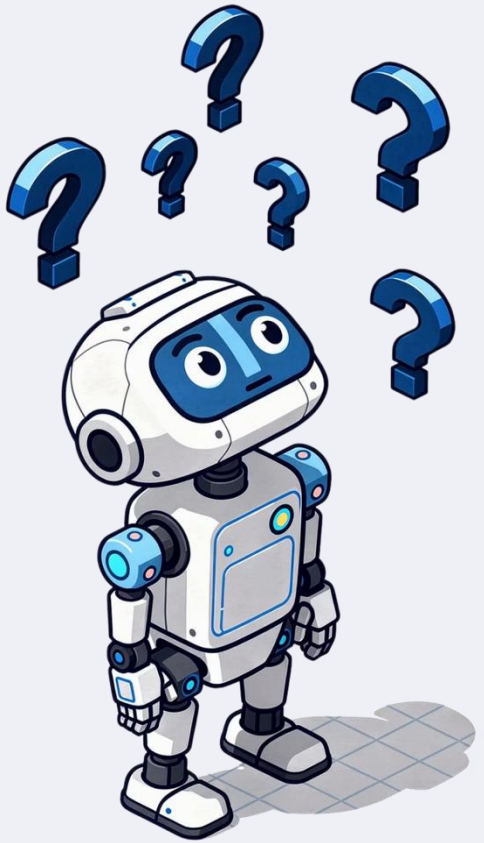
LLMs are trained on data only up to a certain point (**Knowledge Cutoff**) and have no awareness of events or information that emerged after that date.

Hallucination

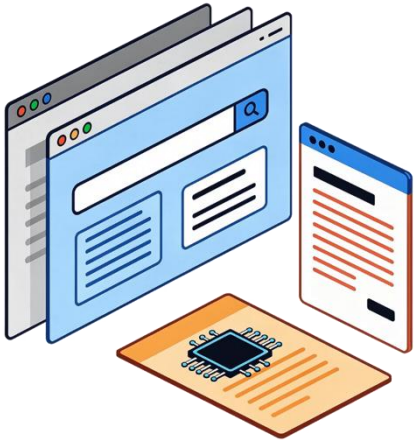
The model generates answers that sound plausible but are factually incorrect, because it has no real source of information to reference.

No Access to Internal Data

LLMs cannot answer questions based on an organization's internal documents or databases, because that data was never part of their training.



What is RAG and Why Use It?

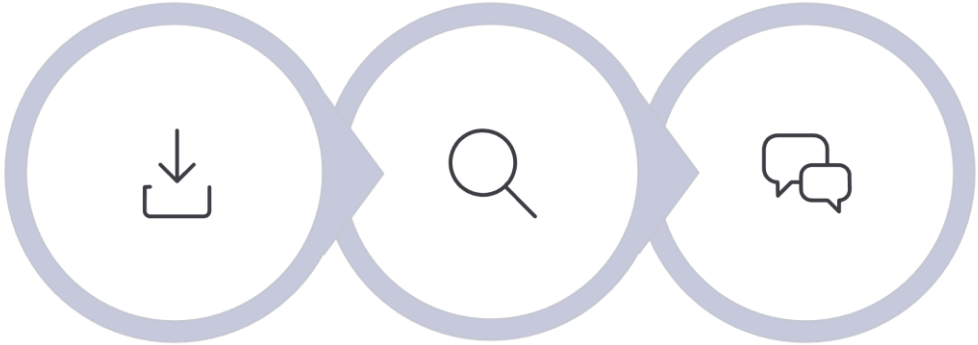


Retrieval-Augmented Generation

A technique that allows LLMs to **"retrieve information first, then respond"** instead of relying solely on the model's memory.

- The system pulls relevant information from a knowledge base and inserts it into the prompt
- Answers are more accurate and can cite their sources
- Knowledge can be updated without retraining the model

RAG Pipeline Overview



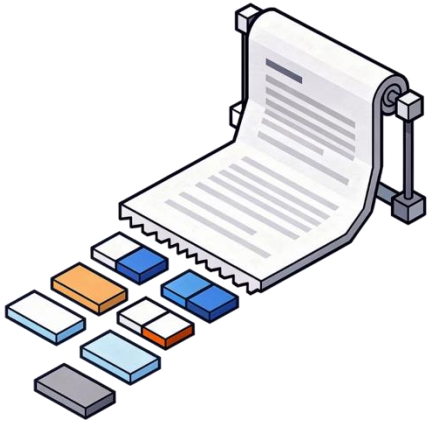
These 3 pipeline stages work together to enable the LLM to answer questions accurately by referencing real, grounded data.

What is Chunking?

Splitting Documents into Smaller Pieces

Chunking is the process of dividing large documents into smaller **"chunks"** to enable more efficient search and processing.
LLMs have a limited context window — they cannot receive an entire document at once. Chunking directly solves this problem.

A good chunk should be meaningful on its own — not too large and not too small.



Types of Chunking

Fixed-size Chunking

Split by a fixed number of characters/tokens, e.g. every 500 tokens — simple, but may cut in the middle of a sentence

Recursive Character Splitting

Split by separators such as paragraph → sentence → word — the **most widely used** method

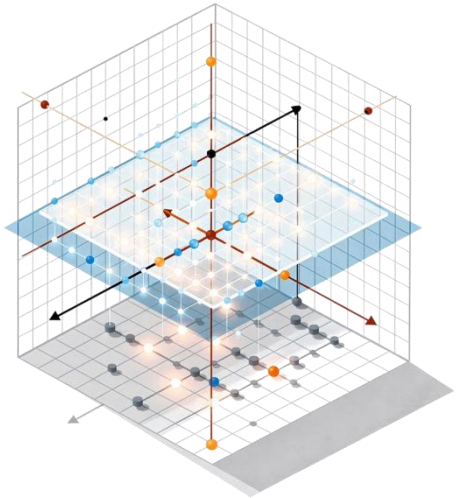
Semantic Chunking

Split by meaning — groups sentences with similar content together

Document-based Chunking

Split by document structure, such as Markdown headers or HTML tags

What is a Vector Database?



A Database Built for Meaning

A Vector Database is designed to store and search **vector embeddings** — multi-dimensional numerical representations of the meaning behind text.

- Unlike SQL which searches by exact match — a vector DB searches by **"semantic similarity"**
- It is the core component of a RAG pipeline in the Retrieval step
- Efficiently handles large datasets of millions of vectors

Example Vector Database Tools

ChromaDB

Open-source, easy to install, ideal for **prototyping and local development** — get started in Python with just a few lines of code

pgvector

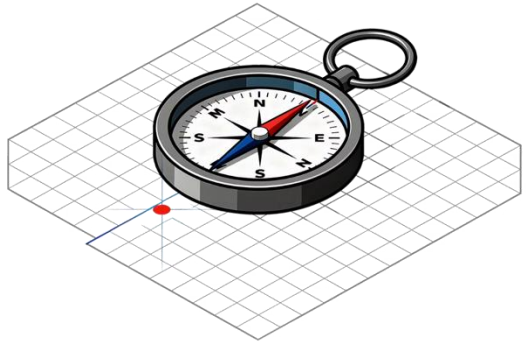
A PostgreSQL extension — adds vector search to your existing database, perfect for **production environments already using Postgres**

Others

Pinecone (managed cloud), Weaviate, Qdrant, Milvus — options for large-scale deployments and specialized use cases

What is Similarity Search?

The process of finding vectors that are **"closest"** to a query vector in vector space — the core of the Retrieval step in RAG



1

Cosine Similarity

Measures the angle between vectors — a value close to 1 = very similar meaning. The **most popular method in NLP**

2

Euclidean Distance (L2)

Measures the straight-line distance between two points in vector space

3

ANN — Approximate Nearest Neighbor

A technique to speed up search in large datasets, such as **HNSW, IVF**

Start Building Your Own RAG

1

Basic Stack

LangChain / LlamaIndex + **ChromaDB** + Gemini Embeddings — get started within a single day

2

First Steps

Choose a document → chunk → embed → store in ChromaDB → test your queries

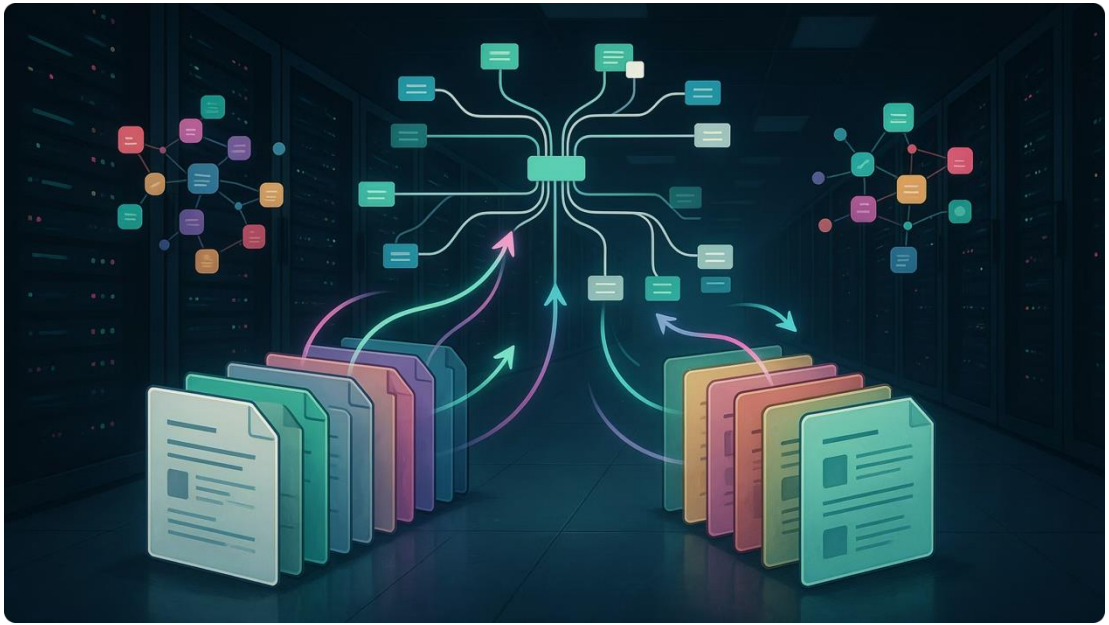
3

Production

Migrate to **pgvector** or Pinecone as you scale, and add re-ranking for improved accuracy

Workshop: Build sample RAG

RAG Pipeline Overview



What is RAG?

Retrieval-Augmented Generation augments an LLM by fetching relevant context from a Vector DB *before* generating a response — reducing hallucination and grounding answers in real data.

Core Pipeline Steps

Key Challenges

- Source data quality upstream
- Embedding model versioning
- Measuring retrieval performance

01

Ingest source documents

02

Chunk into segments

03

Embed with a vector model

04

Store in Vector DB

05

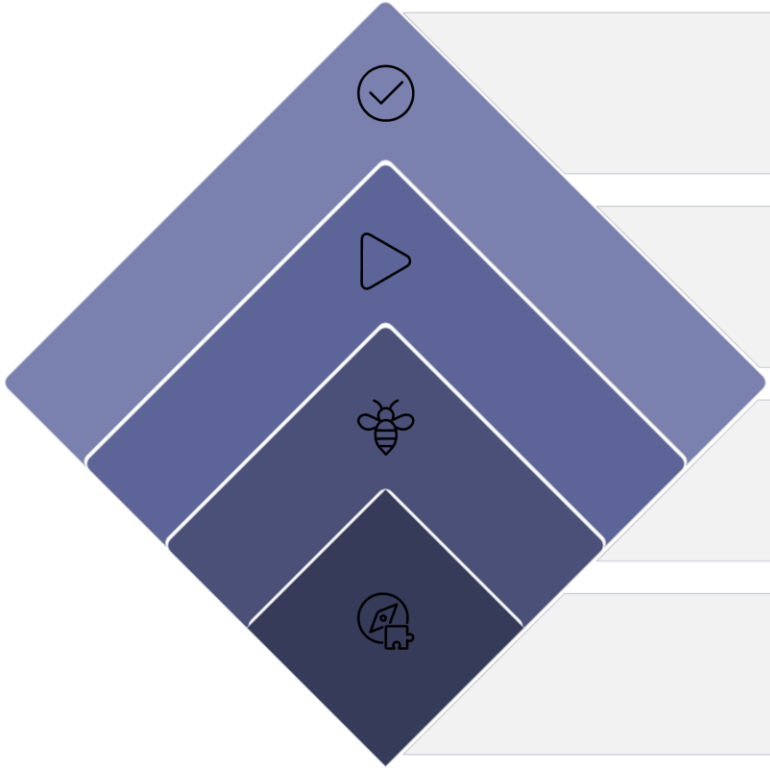
Retrieve on query

06

Generate final response

Workshop: Build RAG pipeline via Airflow

What is an Agent — and How Does It Differ from an LLM?



Evaluate

Check results and decide

Execute

Run the chosen action

Tool Call

Query APIs, DBs, or search

Plan

Select strategy and next action

Unlike a stateless LLM, an Agent wraps the model in a decision loop — giving it **State, Memory, Tools, and the ability to iterate** until a goal is reached.

Key Differences from LLM

- **State & Memory:** Persists context across steps
- **Tool Use:** Can call APIs, search DBs, write files

Concrete Example

An agent tasked with "summarize and alert" will autonomously:

Human-in-the-Loop — Why Is It Necessary?

The Risk of Full Autonomy

Models can **hallucinate**, make wrong decisions, or trigger **irreversible actions** — deleting production data, sending mass emails, making API calls with financial impact.

What HITL Adds

A **human checkpoint** is inserted before any critical action executes — converting autonomous risk into a supervised, auditable decision.

When to Apply It

High-impact outcomes, sensitive or regulated data, and any system still in the **Pilot phase** where model behavior is not yet fully validated.

Workshop: Build AI Agent via Airflow

DataOps for RAG — Upstream Data Quality & Monitoring

DATAOPS · QUALITY GATE

Always Validate Before Chunking

Garbage in, garbage vectors out. Validate source documents at the pipeline entry point before any chunking or embedding occurs.

- Detect **empty files** and zero-byte documents
- Flag **abnormal file sizes** (too small or suspiciously large)
- Catch **garbled characters** from failed PDF/OCR scans

Recommended tool: Integrate **Great Expectations** as the first Task in your Airflow DAG to validate schema and data quality before any processing begins.

DATAOPS · RELIABILITY

Monitoring & Idempotency

A robust RAG pipeline must be **observable and re-runnable** without side effects.

- **Failure Alerts:** Configure Airflow Webhooks to Slack/Teams when Tasks or Sensors fail
- **SLA Tracking:** Monitor task durations to identify bottlenecks before they escalate
- **Idempotency:** Pipelines must be re-runnable without overwriting vectors or losing data
- **Isolation:** Execute each event/day independently to prevent cross-run contamination

MLOps for RAG — Embedding Model Versioning

Why This Breaks Everything

Vector similarity (cosine, dot product) is only meaningful when query vectors and stored document vectors were produced by the **exact same model**. Mixing model versions yields nonsensical similarity scores — and your RAG pipeline returns irrelevant context with no obvious error.

The Engineering Solution

- **Store Model Version in Metadata** alongside every embedded document chunk
- **Re-indexing Pipeline:** Maintain a dedicated DAG ready to rebuild the entire document store when switching models
- Version-gate queries: reject retrieval if query model ≠ stored model version
- Treat model version upgrades as **breaking changes** requiring a migration plan

MLOps for RAG — Retrieval Evaluation & API Monitoring

Hit Rate

Measures whether the correct document chunk appears in the top-k retrieved results. Use **Ragas** to compute this automatically against a golden eval set.

Context Precision

Of all retrieved chunks, how many are actually relevant? High precision means less noise injected into the LLM prompt — and more accurate answers.

Faithfulness

Does the generated answer stay grounded in the retrieved context? Ragas measures this to detect hallucination at the generation step.

API Monitoring

Track **Token Usage** to control LLM inference costs. Monitor **Safety/Guardrails Block rates** from the Gemini API to detect prompt injection attempts or policy violations.

Scaling to Enterprise

Enterprise Vector DB

Migrate from **ChromaDB** (local dev) to production-grade stores: **pgvector**, **Pinecone**, **Qdrant**, or **Vertex AI Search** on GCP — with HA, backups, and access control built in.

Cloud Orchestration

Move from local Airflow to **Google Cloud Composer** (Managed Airflow) — eliminating infrastructure maintenance overhead while gaining auto-scaling and native GCP integrations.

Data Privacy (PDPA)

Add a **PII Redaction Task** at the very start of the ingestion pipeline — before any data is sent to an external embedding or generation API. Non-negotiable for compliance.

Question & Answer

ประเมินความพึงพอใจของผู้เข้ารับการฝึกอบรม
หลักสูตร “MASTERING WORKFLOW ORCHESTRATION: A DATA ENGINEER'S GUIDE TO RAG ON AIRFLOW”